

1 TypeScript Grammar (WIP)

```
Type ::= AnyType
      | UnknownType
      | NeverType
      | VoidType
      | ObjectType
      | StringType
      | NumberType
      | BooleanType
      | SymbolType
      | BigIntType
      | ObjectLiteralType
      | Id
      | "(" Type ")" [bracket]
      > left:
        Type "&" Type
      > Type "|" Type

ObjectLiteralType ::= "{" PropertySignatures "}"
PropertySignature ::= Id ":" Type
                  | Id "?" ":" Type
                  | "readonly" Id ":" Type
                  | "readonly" Id "?" ":" Type
```

Identifier ::= see <https://tc39.es/ecma262/#prod-IdentifierName>

```
TypeAliasDeclaration ::= "type" Id "=" Type ";"
// <: is not real TS syntax, defined here to mirror the checkTypeRelatedTo
// behavior (aka isAssignable)
// https://raw.githubusercontent.com/microsoft/TypeScript/refs/heads/main/src/compiler/
// checker.ts#:~:
// text=is%20truthy.%0A%20%20%20%20%20%20*%0A%20%20%20%20%20function-,checkTypeRelatedTo,-(%0A%
// 20%20%20%20%20%20%20%20%20source%3A%20Type%2C%0A%20%20%20%20%20%20%20%20%20target
IsAssignableTo ::= Type "<:" Type ";"
Stmt ::= TypeAliasDeclaration
      | IsAssignableTo
Pgm ::= List{Stmt, ""}
```

2 TypeScript Type Checking Environment Configuration (WIP)

$\Gamma := \text{Identifier} \mapsto \text{Type}$

3 TypeScript Type Checking Rules (WIP)

$$\text{LookUp} \frac{}{\Gamma, \text{Id} \mapsto \mathbb{T} | \text{Id} \Downarrow \mathbb{T}}$$
$$\text{Definition} \frac{}{\Gamma | \text{type } \text{Id} = \mathbb{T} ; \Downarrow \Gamma, \text{Id} \mapsto \mathbb{T}}$$

Program sequencing and result accumulation

Currently, we evaluate a program as a list of statements, accumulating a list R of results produced by $<:$ checks.

Core assignability ($<:$) rules

We write the judgment $S <: T$ for $\text{isAssignable}(S, T)$.

Sub-Refl $\frac{}{T <: T}$	Number-Refl $\frac{}{\text{number} <: \text{number}}$
Any-Top-L $\frac{}{\text{any} <: T}$	BigInt-Refl $\frac{}{\text{bigint} <: \text{bigint}}$
Any-Top-R $\frac{}{T <: \text{any}}$	Boolean-Refl $\frac{}{\text{boolean} <: \text{boolean}}$
Unknown-Top $\frac{}{S <: \text{unknown}}$	Symbol-Refl $\frac{}{\text{symbol} <: \text{symbol}}$
Unknown-Any $\frac{}{\text{unknown} <: \text{any}}$	Union-R1 $\frac{S <: T_1}{S <: T_1 \mid T_2}$
Unknown-Refl $\frac{}{\text{unknown} <: \text{unknown}}$	Union-R2 $\frac{S <: T_2}{S <: T_1 \mid T_2}$
Never-Bot $\frac{}{\text{never} <: T}$	Union-L $\frac{S_1 <: T \quad S_2 <: T}{S_1 \mid S_2 <: T}$
String-Refl $\frac{}{\text{string} <: \text{string}}$	

Non-derivable judgments. There are no rules for $\text{unknown} <: T$ when $T \notin \{\text{any}, \text{unknown}\}$, nor for $T <: \text{never}$ unless $T = \text{never}$; thus those judgments are not derivable.

Intersection types. For intersection types, we adopt the usual greatest-lower-bound reading: a value of type $S_1 \& S_2$ satisfies both S_1 and S_2 . At the level of assignability, we capture this with the following rule:

$$\text{Inter-L} \frac{S_1 \& S_2 \quad S_1 <: T \quad S_2 <: T}{S_1 \& S_2 <: T}$$

That is, an intersection type is assignable to a target type T exactly when each constituent is assignable to T . Note that we intentionally do *not* add a converse rule of the form $S <: T_1 \& T_2$, since TypeScript's implementation of intersections (via normalization and structural checks) does not behave as a simple greatest lower bound in all cases.

Object structural subtyping

We write object types in the form

$$\{P\}$$

where each property descriptor P is of one of the following forms:

$$x : T \quad | \quad x? : T \quad | \quad \text{readonly } x : T \quad | \quad \text{readonly } x? : T.$$

We use the following meta-level predicates:

- $\text{name}(P)$ is the identifier of a property descriptor.
- $\text{type}(P)$ is its type.
- $\text{readonly}(P)$ is true iff P is declared `readonly`.
- $\text{required}(P)$ is true iff P is not marked with “?”.

Structured object subtyping (ignoring functions and intersections). Let $S = \{\mathbf{P}_s\}$ and $T = \{\mathbf{P}_t\}$. We say that $S <: T$ holds when all of the following conditions are satisfied:

- For every property descriptor $P_t \in \mathbf{P}_t$ with name $x = \text{name}(P_t)$, there exists *some* $P_s \in \mathbf{P}_s$ with $\text{name}(P_s) = x$ such that:
 - If $\text{required}(P_t)$ then $\text{required}(P_s)$.
 - If $\text{readonly}(P_t)$ then $\text{readonly}(P_s)$.
 - $\text{type}(P_s) <: \text{type}(P_t)$.
- No further condition is imposed on properties of S whose names do not appear in T (extra properties on S are allowed).

This can be encoded by the following rule schema:

$$\frac{\forall P_t \in \mathbf{P}_t. \exists P_s \in \mathbf{P}_s. \begin{array}{l} \text{name}(P_s) = \text{name}(P_t) \wedge \\ (\text{required}(P_t) \Rightarrow \text{required}(P_s)) \wedge \\ (\text{readonly}(P_t) \Rightarrow \text{readonly}(P_s)) \wedge \\ (\text{type}(P_s) <: \text{type}(P_t)) \end{array}}{\{\mathbf{P}_s\} <: \{\mathbf{P}_t\}}$$